

Data Structures and Algorithms in Python

Lecture 5: Data Structures

Part 1: Data Structures as ADTs

Operations, Abstractions, and Python Implementations

Faculty of Data Science and Artificial Intelligence

National Economics University

Part 1 roadmap

1. Operations

What do we need to do with the data?

- access, search
- insert, delete, update
- traverse, min/max

2. ADTs

Describe behavior before implementation.

- sequence, map, set
- stack, queue, deque
- tree-based structures

3. Python tools

Select a concrete implementation.

- list, dict, set
- deque, heapq
- explicit node classes

What is a data structure?

Core definition

A data structure organizes data so that particular operations can be performed correctly and efficiently.



Design question

Do not ask only “Where can I store the values?” Ask: “Which operations must be efficient for my workload?”

Common operations on data structures

Operation	Question	Example
Access / retrieve	Which item is at position or key?	$A[i]$, $\text{records}[\text{id}]$
Search	Does a value or key exist?	find a target, membership test
Insert	Add a new item	append, enqueue, insert node
Delete	Remove an item	pop, dequeue, remove node
Update	Change an existing value	change grade for a student ID
Traverse	Visit every item	print all records, sum values
Min / max	Find an extreme item	smallest key, highest priority

Main idea

The same operation can have very different costs under different representations.

ADT: behavior before implementation

Abstract Data Type (ADT)

An ADT specifies the values, the allowed operations, and the required behavior of those operations. It does not specify how the values are stored in memory.

ADT	Operations	Behavior rule
Stack	push, pop, peek	LIFO
Queue	enqueue, dequeue, front	FIFO
Set	add, remove, membership	unique values
Map / dictionary	lookup, insert, update, delete by key	key $\rightarrow$ value

Important distinction

ADT describes behavior

$\neq$

implementation describes representation

For example, a stack may be implemented by an array, a Python list, or a linked list

Common data structures and typical problems

Structure / ADT	Typical operations	Typical problems
Array / list	index access, search, insert/delete, traverse	sequences, tables, ordered collections
Linked list	traversal, link update, insert/delete after known node	dynamic sequences, stack/queue implementations
Dictionary / map Set	lookup by key, insert, update, delete membership, add/remove, union/intersection	ID → record lookup, indexing uniqueness, duplicate detection
Stack	push, pop, peek	undo, function calls, DFS, backtracking
Queue / deque Binary Search Tree	enqueue/dequeue, both-end updates search, insert, delete, ordered traversal	scheduling, BFS, sliding windows dynamic ordered data, range queries

Search-oriented choices

Hash tables support fast exact lookup; sorted arrays and trees preserve order; sequential structures are simple but may require scans.

Advanced data structures: trees, heaps, indexes, and graphs

Structure family	Main idea	Typical problems
Tree	Parent–child hierarchy	file systems, expression trees, organization charts
Binary Tree	Each node has at most two children	recursive algorithms, heaps, search-tree foundations
Binary Search Tree	Left keys $<$ node key $<$ right keys	ordered search, min/max, predecessor/successor, range queries
Balanced BST	Maintain logarithmic height	ordered maps/sets with worst-case $O(\log n)$ operations
Heap / priority queue	Parent priority dominates children	extract-min/max, scheduling, shortest paths
B-tree / B+ tree	Multiway search tree with many keys per node	database and file-system indexes
Trie	Branch by characters or symbols	prefix search, autocomplete, dictionaries
Graph	Vertices connected by edges	networks, routes, dependencies, relationships

Concrete data structures in Python

Built-in containers

- `list`: dynamic array
- `tuple`: immutable sequence
- `dict`: hash-table-based map
- `set`: hash-table-based set
- `frozenset`: immutable set

Standard-library tools

- `collections.deque`: double-ended queue
- `heapq`: heap operations on a list
- `queue`: synchronized queues
- `array`: compact numeric arrays
- `Counter`, `defaultdict`: specialized containers

Learning note

Linked lists, trees, and graph structures are often implemented explicitly with user-defined classes when we study representation and algorithm behavior.

Python examples: same data, different access

Sequence and membership

```
values = [7, 3, 5, 2]
values.append(6)
first = values[0]
found = 5 in values    # linear scan

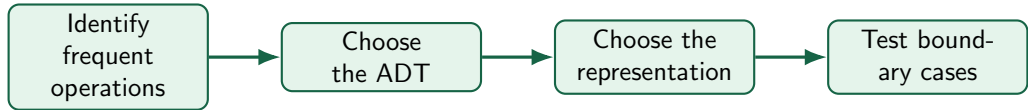
seen = {7, 3, 5, 2}
fast = 5 in seen       # expected O(1)
```

Keyed records and queues

```
grades = {"S01": 8.5}
grades["S02"] = 9.0
score = grades.get("S01")

from collections import deque
q = deque()
q.append("ticket-1")
next_ticket = q.popleft()
```

Design workflow: from workload to implementation



Need fast indexed access

Dynamic array → Python list.

Need FIFO behavior

Queue ADT → deque or linked queue.

Need ordered range queries

Tree-based structure or sorted representation.

Self-check: ADTs and implementations

- ① A program needs to undo the most recent editing command. Which ADT is natural?
- ② A system retrieves student records by student ID. Which Python structure is natural?
- ③ A report needs all IDs in increasing order and range queries such as IDs between 1000 and 1999. Why might an ordered structure be better than a hash set?
- ④ Why is a stack not the same thing as a Python list?
- ⑤ Which advanced structure family is commonly used for database indexes: BST, heap, or B-tree?

Discussion rule

For each answer, name the required operation before naming the data structure.

Key takeaways

- Data structures should be chosen from the operations required by the workload.
- An ADT describes behavior and operations; an implementation describes memory representation.
- Core structures include arrays/lists, linked lists, stacks, queues, dequeues, dictionaries/maps, and sets.
- Advanced structures include tree families, BSTs, balanced BSTs, heaps, B-trees/B+ trees, tries, and graphs.
- Python provides built-in and standard-library containers, but linked structures and trees are often implemented explicitly when needed for learning or custom behavior.
- The design workflow is: identify operations → choose ADT → choose implementation.